

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 1

A string is an ordered sequence of Unicode characters. Strings are immutable, so changing their contents creates a new string. They support indexing, slicing, iteration, concatenation, and methods such as `lower()` and `split()`. Single, double, or triple quotes can delimit a string.

Question 2

```
>>> mystring = "Hello, World!"
>>> print(mystring)
Hello, World!"
```

Question 3

```
>>> mystring = "'Hello, World!'"
>>> print(mystring)
'Hello, World!'"
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 4

The poem has six lines with five newline characters and no newline at the end.

```
>>> poem = """First I saw the white bear, then I saw the black;
... Then I saw the camel with a hump upon his back;
... Then I saw the grey wolf, with mutton in his maw;
... Then I saw the wombat waddle in the straw;
... Then I saw the elephant a waving of his trunk;
... Then I saw the monkeys - mercy, how unpleasantly they smelt!"""
>>> print(poem)
First I saw the white bear, then I saw the black;
Then I saw the camel with a hump upon his back;
Then I saw the grey wolf, with mutton in his maw;
Then I saw the wombat waddle in the straw;
Then I saw the elephant a waving of his trunk;
Then I saw the monkeys - mercy, how unpleasantly they smelt!
```

Question 5

The poem contains 298 characters, including spaces, punctuation, and newlines.

```
>>> len(poem)
298
```

Question 6

There are 31 unique characters. Uppercase and lowercase letters count separately.

```
>>> len(set(poem))
31
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 7

```
>>> import string
>>> poem_nopunc = poem.translate(str.maketrans('', '', string.punctuation))
>>> print(poem_nopunc)
First I saw the white bear then I saw the black
Then I saw the camel with a hump upon his back
Then I saw the grey wolf with mutton in his maw
Then I saw the wombat waddle in the straw
Then I saw the elephant a waving of his trunk
Then I saw the monkeys mercy how unpleasantly they smelt
```

Question 8

```
>>> poem_words = poem_nopunc.split()
>>> print(poem_words)
['First', 'I', 'saw', 'the', 'white', 'bear', 'then', 'I', 'saw', 'the', 'black', 'Then', 'I',
'saw', 'the', 'camel', 'with', 'a', 'hump', 'upon', 'his', 'back', 'Then', 'I', 'saw', 'the',
'grey', 'wolf', 'with', 'mutton', 'in', 'his', 'maw', 'Then', 'I', 'saw', 'the', 'wombat',
'waddle', 'in', 'the', 'straw', 'Then', 'I', 'saw', 'the', 'elephant', 'a', 'waving', 'of',
'his', 'trunk', 'Then', 'I', 'saw', 'the', 'monkeys', 'mercy', 'how', 'unpleasantly', 'they',
'smelt']
```

Question 9

There are 62 words.

```
>>> len(poem_words)
62
```

Question 10

There are 34 unique words, counting capitalization as written.

```
>>> len(set(poem_words))
34
```

Question 11

The 16th word from the end is 'elephant'.

```
>>> poem_words[-16]
'elephant'
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 12

Method 1 uses slicing with a step of -1. Method 2 uses a for loop with reversed(). Both preserve the original list.

```
>>> print(*poem_words[::-1])
smelt they unpleasantly how mercy monkeys the saw I Then trunk his of waving a elephant the saw
I Then straw the in waddle wombat the saw I Then maw his in mutton with wolf grey the saw I
Then back his upon hump a with camel the saw I Then black the saw I then bear white the saw I
First
>>> for word in reversed(poem_words):
...     print(word, end=" ")
...
smelt they unpleasantly how mercy monkeys the saw I Then trunk his of waving a elephant the saw
I Then straw the in waddle wombat the saw I Then maw his in mutton with wolf grey the saw I
Then back his upon hump a with camel the saw I Then black the saw I then bear white the saw I
First >>> print()
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 13

Both methods start with the first word, then output words at indexes 0, 2, 4, and so on.

```
>>> print(*poem_words[::2])
First saw white then saw black I the with hump his Then saw grey with in maw I the waddle the
Then saw elephant waving his Then saw monkeys how they
>>> for i in range(0, len(poem_words), 2):
...     print(poem_words[i], end=" ")
...
First saw white then saw black I the with hump his Then saw grey with in maw I the waddle the
Then saw elephant waving his Then saw monkeys how they >>> print()
```

Question 14

The loop stops before printing mutton.

```
>>> for word in poem_words:
...     if word == "mutton":
...         break
...     print(word, end=" ")
...
First I saw the white bear then I saw the black Then I saw the camel with a hump upon his back
Then I saw the grey wolf with >>> print()
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 15

```
>>> poem_tuple = tuple(poem_words)
>>> print(poem_tuple)
('First', 'I', 'saw', 'the', 'white', 'bear', 'then', 'I', 'saw', 'the', 'black', 'Then', 'I',
'saw', 'the', 'camel', 'with', 'a', 'hump', 'upon', 'his', 'back', 'Then', 'I', 'saw', 'the',
'grey', 'wolf', 'with', 'mutton', 'in', 'his', 'maw', 'Then', 'I', 'saw', 'the', 'wombat',
'waddle', 'in', 'the', 'straw', 'Then', 'I', 'saw', 'the', 'elephant', 'a', 'waving', 'of',
'his', 'trunk', 'Then', 'I', 'saw', 'the', 'monkeys', 'mercy', 'how', 'unpleasantly', 'they',
'smelt')
```

Question 16

Offset 38 is the 39th word because Python indexes start at zero.

```
>>> slice = poem_words[38:42]
>>> print(slice)
['waddle', 'in', 'the', 'straw']
```

Question 17

```
>>> slice[0] = "drink"
>>> print(slice)
['drink', 'in', 'the', 'straw']
```

Question 18

Python sorts strings lexicographically by Unicode code point. Uppercase letters come before lowercase letters in this poem, so First, I, and Then appear before the lowercase words. The list is now sorted for questions 19 and 20.

```
>>> poem_words.sort()
>>> print(poem_words)
['First', 'I', 'I', 'I', 'I', 'I', 'I', 'I', 'Then', 'Then', 'Then', 'Then', 'Then', 'a', 'a',
'back', 'bear', 'black', 'camel', 'elephant', 'grey', 'his', 'his', 'his', 'how', 'hump', 'in',
'in', 'maw', 'mercy', 'monkeys', 'mutton', 'of', 'saw', 'saw', 'saw', 'saw', 'saw', 'saw',
'saw', 'smelt', 'straw', 'the', 'the', 'the', 'the', 'the', 'the', 'the', 'the', 'then',
'they', 'trunk', 'unpleasantly', 'upon', 'waddle', 'waving', 'white', 'with', 'with', 'wolf',
'wombat']
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 19

```
>>> h_words = [word for word in poem_words if word.startswith("h")]
>>> print(h_words)
['his', 'his', 'his', 'how', 'hump']
```

Question 20

```
>>> m_words = [word.upper() if "m" in word else word for word in poem_words]
>>> print(m_words)
['First', 'I', 'I', 'I', 'I', 'I', 'I', 'I', 'Then', 'Then', 'Then', 'Then', 'Then', 'a', 'a',
'back', 'bear', 'black', 'CAMEL', 'elephant', 'grey', 'his', 'his', 'his', 'how', 'HUMP', 'in',
'in', 'MAW', 'MERCY', 'MONKEYS', 'MUTTON', 'of', 'saw', 'saw', 'saw', 'saw', 'saw', 'saw',
'saw', 'SMELT', 'straw', 'the', 'the', 'the', 'the', 'the', 'the', 'the', 'the', 'then',
'they', 'trunk', 'unpleasantly', 'upon', 'waddle', 'waving', 'white', 'with', 'with', 'wolf',
'WOMBAT']
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 21

The table contains products for 1 through 12. With zero-based list indexes, `multi_table[i - 1][j - 1]` equals $i * j$. Row 2 correctly includes 10.

```
>>> multi_table = [[i * j for j in range(1, 13)] for i in range(1, 13)]
>>> for row in multi_table:
...     print(row)
...
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]
[3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36]
[4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48]
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60]
[6, 12, 18, 24, 30, 36, 42, 48, 54, 60, 66, 72]
[7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84]
[8, 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96]
[9, 18, 27, 36, 45, 54, 63, 72, 81, 90, 99, 108]
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
[11, 22, 33, 44, 55, 66, 77, 88, 99, 110, 121, 132]
[12, 24, 36, 48, 60, 72, 84, 96, 108, 120, 132, 144]
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 22

```
>>> poem_lower = poem_nopunc.lower()
>>> print(poem_lower)
first i saw the white bear then i saw the black
then i saw the camel with a hump upon his back
then i saw the grey wolf with mutton in his maw
then i saw the wombat waddle in the straw
then i saw the elephant a waving of his trunk
then i saw the monkeys  mercy how unpleasantly they smelt
```

Question 23

The dictionary includes every character in the lowercased, punctuation-free poem, including spaces and newlines.

```
>>> char_counts = {char: poem_lower.count(char) for char in sorted(set(poem_lower))}
>>> print(char_counts)
{'\n': 5, ' ': 57, 'a': 21, 'b': 4, 'c': 4, 'd': 2, 'e': 26, 'f': 3, 'g': 2, 'h': 24, 'i': 17,
'k': 4, 'l': 8, 'm': 8, 'n': 16, 'o': 7, 'p': 4, 'r': 6, 's': 15, 't': 27, 'u': 5, 'v': 1, 'w':
17, 'y': 5}
```

Pair Exercise 2: Strings, Lists, and Dictionaries

Diego Estuar | Partner: Amir Khan | IST 303 | September 30, 2026

Question 24

The loop prints only the six requested vowels. The final line is the single-line extra-credit solution.

```
>>> for key, value in char_counts.items():
...     if key in "aeiouy":
...         print(key, value)
...
a 21
e 26
i 17
o 7
u 5
y 5
>>> print({key: value for key, value in char_counts.items() if key in "aeiouy"})
{'a': 21, 'e': 26, 'i': 17, 'o': 7, 'u': 5, 'y': 5}
```

Question 25

The missing letters are j, q, x, and z. Set subtraction finds them without a loop, an if statement, or the in operator; sorted() puts the answer in alphabetical order.

```
>>> missing_letters = set(string.ascii_lowercase) - set(poem_lower)
>>> print(sorted(missing_letters))
['j', 'q', 'x', 'z']
```